ROS 2 Humble: Computation Graph Concepts and Implementation of Custom-Interface Publisher–Subscriber Systems

Driverless Taskphase – Task 3 Report

Ambrish Singh
B.Tech (Computer Science), First Year, Section CB
Manipal Institute of Technology, Manipal, India
ambrish.mitmpl2026@learner.manipal.edu

***Abstract*—This report documents the installation of ROS 2 Humble Hawksbill on Ubuntu 22.04 (running under WSL 2) and summarizes the core concepts of the ROS 2 computation graph: nodes, topics, services, actions and parameters, together with custom interfaces, the colcon build system, workspace structure and Quality-of-Service (QoS) policies. Two applications were implemented in Python using rclpy. The first publishes a custom TaskOne message containing a wheel's angular velocity and radius, from which a subscriber computes the longitudinal speed v = ωr. The second is a three-node pipeline that uses a custom RT2String message to check whether user-entered strings are palindromes and returns a "Yes"/"No" verdict. Both systems were built with colcon and verified with terminal output and rqt_graph.**

***Index Terms*—ROS 2, Humble, DDS, publish–subscribe, services, custom interfaces, colcon, QoS, rqt_graph.**

I. Introduction

A driverless race car is not one program but many cooperating programs: camera and LiDAR drivers, cone detection, state estimation, path planning and vehicle control. These programs must exchange data reliably and with low latency. The Robot Operating System 2 (ROS 2) is an open-source middleware framework that provides exactly this communication layer, together with build tools, introspection tools and a large ecosystem of reusable packages [6].

The goal of this task was to become proficient with ROS 2 Humble by completing the official beginner CLI and client-library tutorials [2], [3] and then implementing two small systems. Section II describes the environment. Sections III–VI cover the computation graph, custom interfaces, colcon and workspaces, and QoS. Sections VII and VIII explain the two solutions with their ROS graphs. Section IX lists problems faced and Section X concludes.

II. Environment Setup

Ubuntu 22.04 LTS was installed through the Windows Subsystem for Linux (WSL 2) on Windows 11; WSLg allows Linux GUI applications such as turtlesim, rqt and rqt_graph to open as normal windows. ROS 2 Humble was installed from the official deb packages (ros-humble-desktop and ros-dev-tools) following [1]. The line source /opt/ros/humble/setup.bash was added to ~/.bashrc so that every terminal loads ROS 2 automatically, and ROS_DOMAIN_ID was set so that ROS 2 traffic stays isolated from other users on the shared campus network. The installation was verified with the demo talker/listener nodes and turtlesim.

III. The ROS 2 Computation Graph

The ROS 2 graph is the network of ROS 2 elements processing data together at the same time: the nodes and all the communication links between them. Communication is implemented on top of the Data Distribution Service (DDS) standard [7] through a middleware abstraction layer (rmw); Humble uses eProsima Fast DDS by default. Unlike ROS 1, there is no central master: nodes that share the same domain ID discover each other automatically. The graph can be inspected with the ros2 command-line tools (ros2 node, topic, service, action, param) and visualised with rqt_graph.

A. Nodes

A node is a single, modular process responsible for one job, for example a camera driver or a path planner. A node can publish and subscribe to topics, provide or call services, serve or send action goals, and hold parameters. In turtlesim, `/turtlesim` draws and moves the turtle while `/teleop_turtle` reads the keyboard. Node names must be unique: running two keyboard nodes produced a "nodes that share an exact name" warning, which was fixed by remapping the name with `--ros-args --remap __node:=keyboard1`. Remapping can also redirect topic names, which was used to drive a second turtle with `turtle1/cmd_vel:=turtle2/cmd_vel`.

B. Topics

A topic is a named, strongly typed bus over which nodes exchange messages using the publish–subscribe pattern. Publishers and subscribers are anonymous to each other, a topic may have any number of each, and data flows one way, typically as a continuous stream. For example, `/turtle1/cmd_vel` carries `geometry_msgs/msg/Twist` velocity commands from the keyboard node to turtlesim, while `/turtle1/pose` was measured with `ros2 topic hz` at about 62 Hz. The tools `ros2 topic list -t`, `echo`, `info` and `pub` were used to inspect and inject messages.

C. Services

A service follows the request–response pattern: a client sends a request and a single server returns exactly one response. Service types are defined in .srv files with the request fields

above a `---` separator and the response fields below it. The turtlesim services `/spawn`, `/clear`, `/reset` and `/turtle1/set_pen` were called with `ros2 service call`; for instance, /spawn returned the name of the newly created turtle.

D. Actions

Actions are intended for long-running tasks. A client sends a *goal*, the server streams periodic *feedback* while working and finally returns a *result*; the goal can also be cancelled. An action is built from three services (send goal, cancel goal, get result) and two topics (feedback and status). Sending the goal θ = 1.57 rad to `/turtle1/rotate_absolute` produced a stream of "remaining" feedback values decreasing towards zero, followed by the result delta = 2.43 rad and the status SUCCEEDED. The hidden `_action/feedback` and `_action/status` topics were also visible in rqt_graph.

E. Parameters

A parameter is a configuration value that belongs to a node (bool, integer, double, string, byte array, or arrays of these). Parameters can be read and changed at run time without editing code, saved to a YAML file and reloaded. Using `ros2 param set /turtlesim background_r 150` changed the background colour live; `ros2 param dump` saved the settings and `--params-file` started a node with them. In a race car, values such as a maximum speed or a detection threshold are typical parameters.

F. Topic vs. Service: When to Use Each

Table I compares the two mechanisms. A topic should be used for data that is produced continuously and where the newest value matters more than a confirmation, such as sensor readings, odometry, detected cones or velocity commands. A service should be used for occasional operations that need an answer or a confirmation, such as resetting a map, spawning an object or querying a state, and that complete quickly. If the operation takes a long time and progress or cancellation matters (for example, "drive to this waypoint"), an action is the right choice.

TABLE I
COMPARISON OF TOPICS AND SERVICES

Aspect	Topic	Service
Pattern	Publish–subscribe	Request–response
Direction	One-way	Two-way (request, then reply)
Timing	Continuous or periodic stream	On demand; one reply per request
Coupling	Anonymous, many-to-many	One server, many clients
Confirmation	None (fire and forget)	Response confirms completion
Interface	.msg	.srv (request --- response)
Example	/turtle1/cmd_vel, /taskone	/spawn, /clear

IV. CUSTOM INTERFACES

When standard packages such as `std_msgs` or `geometry_msgs` do not describe the data well, custom interfaces are defined: .msg files for messages, .srv files for services (request --- response) and .action files for actions (goal --- result --- feedback) [5]. Interfaces are placed in a dedicated `ament_cmake` package; `rosidl_generate_interfaces()` in CMakeLists.txt generates the Python and C++ code, and package.xml declares `rosidl_default_generators` as a build tool, `rosidl_default_runtime` as an execution dependency and membership of `rosidl_interface_packages`.

For this task a package named `task_interfaces` was created with the files below. ROS 2 requires type names in CamelCase but field names in lowercase snake_case, so the field *angVel* requested in the task was named `ang_vel`; a capital letter in a field name causes the build to fail. A Palindrome.srv service was also defined for an alternative Q2 design (Section VIII-E).

```
# msg/TaskOne.msg
float64 ang_vel
float64 radius
# msg/RT2String.msg
string data
# srv/Palindrome.srv
int32 result
---
string answer
```

After building, `ros2 interface show task_interfaces/msg/TaskOne` confirmed the generated type, and other packages import it with `from task_interfaces.msg import TaskOne`.

V. COLCON AND WORKSPACE STRUCTURE

colcon ("collective construction") is the ROS 2 build tool [8]. It finds all packages in a workspace, orders them by their declared dependencies and builds each one with the right build type: ament_cmake for C++ and interface packages, ament_python for Python packages. A workspace has the layout shown below; only src is written by the developer, while colcon creates build (intermediate files), install (the runnable result plus setup scripts) and log.

```
ros2_ws/
├── src/
│   ├── task_interfaces/   (ament_cmake)
│   │   ├── msg/  srv/
│   │   └── CMakeLists.txt, package.xml
│   ├── task_one/          (ament_python)
│   │   ├── task_one/publisher.py, subscriber.py
│   │   └── setup.py, package.xml
│   └── task_two/          (ament_python)
│       └── task_two/node1.py, node2.py, node3.py
├── build/   ├── install/   └── log/
```

The ROS 2 installation in /opt/ros/humble is the *underlay*; the workspace is an *overlay* sourced on top of it, and a package in the overlay takes priority over a package with the same name in

the underlay. `install/setup.bash` sources the underlay and the overlay, whereas `local_setup.bash` sources only the overlay. For Python packages, the `console_scripts` entry points in setup.py register the executables so that `ros2 run task_one publisher` works. Useful options were `--symlink-install` (edited Python files take effect without reinstalling), `--packages-select` (build only chosen packages) and `--parallel-workers` (limit parallel builds).

VI. Quality of Service (QoS) Settings

QoS policies control how DDS delivers messages, letting a developer trade reliability against latency and memory [4]. The main policies are summarised in Table II. ROS 2 also provides predefined profiles: the default profile (keep last 10, reliable, volatile), the sensor-data profile (best effort with a small queue, for high-rate streams where only the newest sample matters), and profiles for services and parameters.

TABLE II
Main QoS Policies

Policy	Options	Effect
History	KEEP_LAST, KEEP_ALL	Whether only the last N samples or all samples are stored
Depth	Integer N	Queue size used with KEEP_LAST
Reliability	RELIABLE, BEST_EFFORT	Retransmit until delivered, or send once and accept loss
Durability	VOLATILE, TRANSIENT_LOCAL	Whether late-joining subscribers receive previously published samples
Deadline	Duration	Maximum expected time between messages
Lifespan	Duration	Age after which a message is discarded
Liveliness	AUTOMATIC, MANUAL_BY_TOPIC + lease	How a publisher signals that it is still alive

A publisher and a subscriber only connect if their policies are compatible ("requested vs. offered"). For example, a RELIABLE subscriber cannot receive from a BEST_EFFORT publisher, and a TRANSIENT_LOCAL subscriber cannot match a VOLATILE publisher; in such cases no data flows and ROS 2 reports an incompatible-QoS warning. In both solutions the integer 10 was passed to create_publisher and create_subscription, which selects the default profile with KEEP_LAST depth 10, RELIABLE and VOLATILE. This suits user-entered data at low rates, where no value should be lost. On a real car, camera and LiDAR topics would use the sensor-data profile, while a static map or configuration would use TRANSIENT_LOCAL so that nodes started later still receive it.

VII. Solution 1: Wheel Longitudinal Speed (Q1)

A. Design

The system uses two packages: `task_interfaces` (the TaskOne message) and `task_one` (two Python nodes). The node `/taskone_publisher` reads the angular velocity and wheel radius from the terminal and publishes them on the topic `/taskone`; the node `/taskone_subscriber` receives each message and computes the speed. A topic is used because the data only flows one way and no reply is needed.

B. Physics

For a wheel rolling without slipping, one revolution moves the wheel centre forward by one circumference $2\pi r$. Hence the longitudinal (forward) speed of the wheel is

$$v = \omega \cdot r \tag{1}$$

where ω is the angular velocity in rad/s, r is the radius in m and v is in m/s.

C. Publisher Node

The publisher is created with `create_publisher(TaskOne, '/taskone', 10)`. In a loop it reads both values with `input()`, converts them to float (a ValueError prints a warning instead of crashing), fills a TaskOne message and publishes it. Because this node only sends data, it does not need to call spin(); Ctrl+C shuts it down cleanly.

D. Subscriber Node

The subscriber is created with `create_subscription(TaskOne, '/taskone', self.callback, 10)` and kept alive with `rclpy.spin()`. Every time a message arrives, the callback computes `speed = msg.ang_vel * msg.radius` using (1) and prints the result with the node logger.

E. Results

Both packages were built with colcon build --packages-select task_interfaces task_one. With $\omega = 10$ rad/s and $r = 0.3$ m, the subscriber printed a longitudinal speed of 3.00 m/s, matching (1) (Fig. 1). The rqt_graph in Fig. 2 shows the publisher node connected to the subscriber node through the /taskone topic.

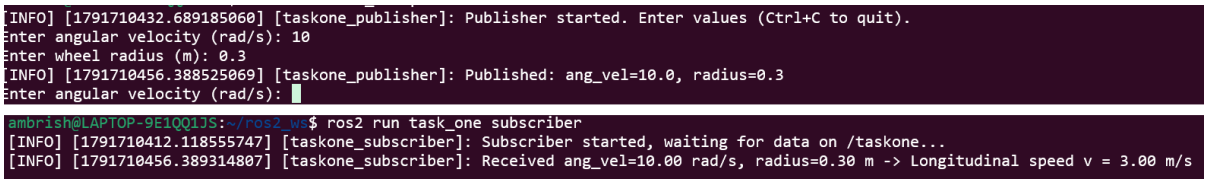


Fig. 1. Q1 terminal output. Top: the publisher reads ω = 10 rad/s and r = 0.3 m and publishes them. Bottom: the subscriber computes v = 3.00 m/s.



Fig. 2. rqt_graph for Q1: /taskone_publisher → /taskone (TaskOne) → /taskone_subscriber.

VIII. Solution 2: Palindrome Checker (Q2)

A. Design

The package `task_two` contains three nodes that use the RT2String message from `task_interfaces` and the standard Int32 and String messages. The topics are listed in Table III.

TABLE III
Topics Used in Q2

Topic	Type	Publisher → Subscriber
/rt2string	RT2String	node1 → node2 (user string)
/palindrome_result	std_msgs/Int32	node2 → node3 (1 or 0)
/palindrome_answer	std_msgs/String	node3 → node2 (“Yes”/“No”)

B. Node 1: String Publisher

Node 1 (`/node1_string_publisher`) publishes the most recently entered string on `/rt2string` once per second using a ROS 2 timer, so the string is published continuously until the user types a new one (e.g. “AABAA”, then “BACAB”). Keyboard input runs in a separate thread so that waiting for input never blocks the timer, and a lock protects the shared string.

C. Node 2: Palindrome Checker

Node 2 (`/node2_palindrome_checker`) subscribes to `/rt2string`. Its callback removes spaces, converts the string to lowercase and compares it with its reverse (`s == s[::-1]`). It publishes 1 if the string is a palindrome and 0 otherwise on `/palindrome_result`. It also subscribes to `/palindrome_answer` and prints Node 3’s reply together with the string it refers to.

D. Node 3: Answer Node

Node 3 (`/node3_answer`) subscribes to `/palindrome_result`, maps 1 to “Yes” and 0 to “No”, and publishes the answer back to Node 2 on `/palindrome_answer`.

E. Design Choice: Topics Instead of a Service

The exchange between Node 2 and Node 3 is a request and a reply, so a service (Palindrome.srv) would also be a natural fit and would guarantee that each reply belongs to exactly one request. However, rqt_graph in ROS 2 Humble does not display service connections, so Node 3 would appear disconnected in the required graph. Two topics were therefore used so that the complete data flow is visible. The trade-off is that topics have no built-in request–reply pairing; Node 2 associates each answer with the last string it processed, which is sufficient here because strings arrive at 1 Hz and processing takes only milliseconds.

F. Results

The package was built with colcon build --packages-select task_two, and the three nodes were started in separate terminals. For the input “AABAA”, Node 2 sent 1 and Node 3 replied “Yes”; for “HELLO”, Node 2 sent 0 and Node 3 replied “No” (Fig. 3). The output repeats every second because Node 1 publishes continuously. The rqt_graph in Fig. 4 shows the full loop: node1 → /rt2string → node2 → /palindrome_result → node3 → /palindrome_answer → node2.

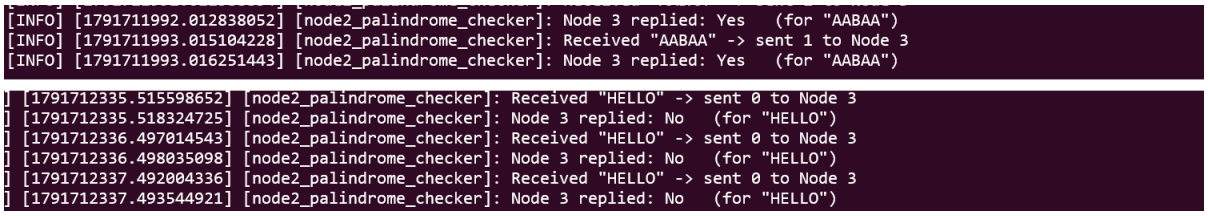


Fig. 3. Q2 output of Node 2. Top: “AABAA” is a palindrome, so 1 is sent and Node 3 replies “Yes”. Bottom: “HELLO” is not, so 0 is sent and Node 3 replies “No”.

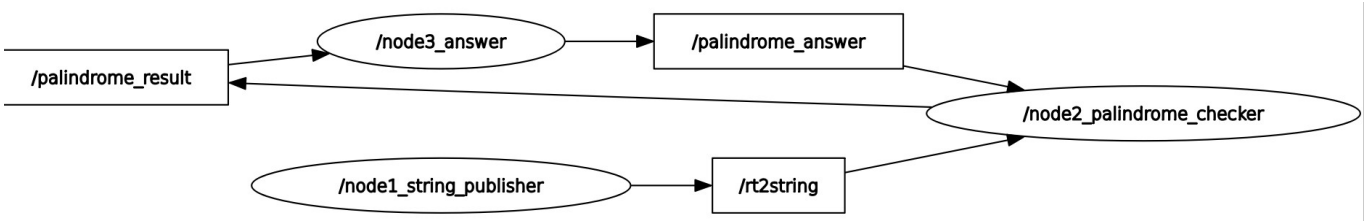


Fig. 4. rqt_graph for Q2: node1 publishes on /rt2string; node2 publishes the result (1/0) on /palindrome_result to node3, which replies on /palindrome_answer.

IX. Challenges and Lessons Learned

Build freezing in WSL: building all 22 example packages in parallel exhausted memory and colcon stopped making progress. Building only the needed packages with `--packages-select`, limiting parallelism with `--parallel-workers 2` and `MAKEFLAGS=-j2`, and adding a COLCON_IGNORE file to the examples folder solved it.

Duplicate node names: two teleop nodes with the same name caused a warning and confusing behaviour; remapping `__node` gave each a unique name.

Interface naming rules: the field name angVel is not allowed, so ang_vel was used.

Duplicate node instances: during Q2 testing every result was printed twice. `ros2 node list` showed that Node 3 had been started in two terminals; stopping one fixed it.

Sourcing: a newly built package is not found by ros2 run until `install/setup.bash` is sourced in that terminal.

X. Conclusion

ROS 2 Humble was installed on Ubuntu 22.04 under WSL 2, and the beginner CLI and client-library tutorials were completed. The work covered the computation graph and its building blocks (nodes, topics, services, actions and parameters), custom interfaces, the colcon workspace structure and QoS policies. Two systems were implemented with custom messages: a publisher–subscriber pair that computes a wheel’s longitudinal speed from its angular velocity and radius, and a three-node palindrome checker with a feedback loop between two nodes. Both were verified through terminal output and rqt_graph. These are the same patterns used in a driverless pipeline, where perception, planning and control nodes exchange data over topics with suitable QoS.

Code availability: the packages task_interfaces, task_one and task_two and the rqt_graph screenshots are in the folder Task3 of the GitHub repository `https://github.com/Ambrish490/Ambrish_Driverless_Taskphase`.

References

[1] Open Robotics, “Ubuntu (deb packages),” *ROS 2 Documentation: Humble*. [Online]. Available: https://docs.ros.org/en/humble/Installation/Ubuntu-Install-Debs.html

[2] Open Robotics, “Beginner: CLI tools,” *ROS 2 Documentation: Humble*. [Online]. Available: https://docs.ros.org/en/humble/Tutorials/Beginner-CLI-Tools.html

[3] Open Robotics, “Beginner: Client libraries,” *ROS 2 Documentation: Humble*. [Online]. Available: https://docs.ros.org/en/humble/Tutorials/Beginner-Client-Libraries.html

[4] Open Robotics, “Quality of Service settings,” *ROS 2 Documentation: Humble*. [Online]. Available: https://docs.ros.org/en/humble/Concepts/Intermediate/About-Quality-of-Service-Settings.html

[5] Open Robotics, “Interfaces,” *ROS 2 Documentation: Humble*. [Online]. Available: https://docs.ros.org/en/humble/Concepts/Basic/About-Interfaces.html

[6] S. Macenski, T. Foote, B. Gerkey, C. Lalancette, and W. Woodall, “Robot Operating System 2: Design, architecture, and uses in the wild,” *Science Robotics*, vol. 7, no. 66, art. eabm6074, May 2022.

[7] Object Management Group, *Data Distribution Service (DDS), Version 1.4*, OMG Standard, Apr. 2015.

[8] colcon contributors, “colcon – collective construction,” documentation. [Online]. Available: https://colcon.readthedocs.io